Task 2: Student Management System (CLI)

1. Goal

You will build a **command-line (CLI) application** that manages students, instructors and classes. When you run the program, it shows a prompt and waits for you to type commands. Each command does one job (add a student, list the classes, and so on), and the program keeps running until you type `exit`.

All the data lives **only in memory** during the current run. There is no database and no file saving: when the program closes, the data is gone. That is expected.

This is your first CLI program, so this document describes every command in detail: what you type, what the program asks, what it prints, and which errors it must handle.

2. What is a CLI application?

A CLI application has no windows or buttons. It repeats the same cycle:

1. Print a prompt (here `sms>`).
2. Read one line typed by the user.
3. Work out which command it is.
4. Run that command and print the result or an error message.
   [date]
5. Go back to step 1, until the user types `exit`.

This cycle is called the **main loop** (or REPL: read, evaluate, print, loop). Your program is mostly this loop plus one function per command.

3. The three kinds of data

You decide how to design your structs and containers. The program must manage three kinds of records:

Record	Information stored
Student	ID, full name, age, email
Instructor	ID, full name, department
Class	ID, class name, instructor, capacity (maximum number of students), the list of enrolled students

Rules for the data:

- **IDs are generated by the program**, not typed by the user. Each kind has its own counter that starts at 1 and goes up by 1 for each new record (the first student is student 1, the first instructor is instructor 1, the first class is class 1).
- **Instructor names must be unique** (ignoring upper/lower case), because a class is linked to its instructor by name.
- A class always has exactly one instructor, who must already exist.
- A student can be enrolled in many classes. A class holds up to its capacity.

4. Program start and the main loop

When started, the program prints a welcome banner, then shows the prompt `sms>` and waits.

```
==========================================
 Student Management System   (type 'help')
==========================================
sms>
```

Rules for reading input:

- Read the **whole line** the user typed, then split it into words.
- Commands are **not case sensitive**: `LIST STUDENTS`, `List Students` and `list students` do the same thing.
- Extra spaces before, between or after words are ignored.
- An **empty line** does nothing: just show the prompt again.
- An unknown command prints the message below and the loop continues. The program must **never crash** because of what the user types.

```
sms> hello world
Error: unknown command 'hello world'. Type 'help' to see the available commands.
```

5. The commands

There are 10 commands. Details of each one follow.

Command	Purpose
`help`	Show the list of commands
`add student`	Add a new student
`add instructor`	Add a new instructor
`create class`	Create a class taught by an existing instructor

Command	Purpose
`list students`	Show all students
`list instructors`	Show all instructors
`list classes`	Show all classes
`enroll <student_id> <class_id>`	Enroll a student in a class
`show class <class_id>`	Show the details of one class and its students
`exit`	Quit the program

5.1 `help`

What it does: prints every command with a one-line description. It takes no arguments.

```
sms> help
Available commands:
  help                          Show this list
  add student                   Add a new student
  add instructor                Add a new instructor
  create class                  Create a new class
  list students                 List all students
  list instructors              List all instructors
  list classes                  List all classes
  enroll <student_id> <class_id>    Enroll a student in a class
  show class <class_id>         Show one class with its students
  exit                          Quit the program
```

5.2 `add student`

What it does: asks the user for the student's data, one question per line, then stores a new student with the next free student ID.

Questions asked, in this order:

1. `Full name:` (may contain spaces, so read the whole line)
2. `Age:`
3. `Email:`

Validation:

- The name must not be empty.
- The age must be a whole number between 15 and 100.
- The email must contain an `@` character.

- When an answer is invalid, print an error and **cancel the command** (nothing is added), then return to the main prompt.

Output on success: `Student added successfully (ID: <id>).`

```
sms> add student
Full name: Ali Hassan
Age: 20
Email: ali@uni.edu
Student added successfully (ID: 1).

sms> add student
Full name: Mona Samir
Age: abc
Error: age must be a whole number between 15 and 100. Student not added.
```

5.3 `add instructor`

What it does: asks for the instructor's data and stores a new instructor with the next free instructor ID.

Questions asked, in this order:

1. `Full name:`
2. `Department:`

Validation:

- The name and the department must not be empty.
- The name must not already belong to another instructor (ignore upper/lower case).

Output on success: `Instructor added successfully (ID: <id>).`

```
sms> add instructor
Full name: Dr. Sara Nabil
Department: Computer Science
Instructor added successfully (ID: 1).

sms> add instructor
Full name: dr. sara nabil
Department: Mathematics
Error: an instructor named 'dr. sara nabil' already exists. Instructor not added.
```

5.4 `create class`

What it does: creates a new class and links it to an existing instructor.

Questions asked, in this order:

1. `Class name:`
2. `Instructor name:` (the full name of an instructor that was already added; ignore upper/lower case)
3. `Capacity:`

Validation:

- The class name must not be empty and must not already be used by another class.
- The instructor must exist. If not, print the error below and cancel the command.
- The capacity must be a whole number between 1 and 200.

Output on success: `Class created successfully (ID: <id>)`. A new class starts with **no** students enrolled.

```
sms> create class
Class name: Programming 101
Instructor name: Dr. Sara Nabil
Capacity: 2
Class created successfully (ID: 1).

sms> create class
Class name: Databases
Instructor name: Dr. Omar
Error: instructor 'Dr. Omar' not found. Add the instructor first. Class not created.
```

5.5 `list students`

What it does: prints all students in the order they were added, in a table with a header line. If there are no students, print `No students found.`

Every student shows ID, name, age and email. Keep the columns aligned (the header `<iomanip>` tools such as `setw` help with this).

```
sms> list students
ID   Name         Age   Email
---  -----------  ----  ---------------
1    Ali Hassan   20    ali@uni.edu
2    Mona Samir   19    mona@uni.edu
Total: 2 student(s)
```

5.6 `list instructors`

What it does: same idea for instructors. Columns: ID, name, department, number of classes taught. If there are none, print `No instructors found.`

```
sms> list instructors
ID   Name          Department       Classes
---  ------------  ---------------  -------
```

```
1    Dr. Sara Nabil   Computer Science   1
Total: 1 instructor(s)
```

5.6.1 `list classes`

What it does: prints all classes. Columns: ID, class name, instructor name, and enrollment shown as `enrolled/capacity`. If there are none, print `No classes found.`

```
sms> list classes
ID   Class Name       Instructor       Enrolled
---  ---------------  ---------------  --------
1    Programming 101   Dr. Sara Nabil   2/2
Total: 1 class(es)
```

5.7 `enroll <student_id> <class_id>`

What it does: this is the first command that takes **arguments on the same line**. It adds the student to the class. Example: `enroll 1 1` enrolls student 1 in class 1.

Checks, in this order (stop at the first one that fails, print its error, change nothing):

Problem	Message
Wrong number of arguments	`Usage: enroll <student_id> <class_id>`
An argument is not a number	`Error: IDs must be whole numbers.`
No student with that ID	`Error: student with ID <id> not found.`
No class with that ID	`Error: class with ID <id> not found.`
Student already in this class	`Error: <student name> is already enrolled in <class name>.`
Class is full	`Error: class '<class name>' is full (<enrolled>/<capacity>).`

Output on success: `<student name> was enrolled in <class name>.`

```
sms> enroll 1 1
Ali Hassan was enrolled in Programming 101.
sms> enroll 1 1
Error: Ali Hassan is already enrolled in Programming 101.
sms> enroll 2 1
Mona Samir was enrolled in Programming 101.
sms> enroll 5 1
Error: student with ID 5 not found.
sms> enroll x 1
Error: IDs must be whole numbers.
sms> enroll 1
Usage: enroll <student_id> <class_id>
```

If a third student were enrolled in the class above, it would print `Error: class 'Programming 101' is full (2/2).`

5.8 `show class <class_id>`

What it does: prints the full details of one class, including the instructor and every enrolled student. It is the way to check that `enroll` worked.

Errors:

- Missing ID: `Usage: show class <class_id>`
- ID not a number: `Error: IDs must be whole numbers.`
- No class with this ID: `Error: class with ID <id> not found.`

```
sms> show class 1
Class ID    : 1
Name        : Programming 101
Instructor  : Dr. Sara Nabil (Computer Science)
Enrolled    : 2/2
Students    :
  1. [ID 1] Ali Hassan   - ali@uni.edu
  2. [ID 2] Mona Samir   - mona@uni.edu
```

If nobody is enrolled yet, print `  (no students enrolled)` under `Students    :`.

5.9 `exit`

What it does: prints `Goodbye!` and ends the program. `quit` must work as a second name for `exit`. All data is lost, which is fine.

6. Rules

1. Use **structs** for the Student, Instructor and Class records.
2. Write **one function per command** (for example, a function for `add student`, one for `list classes`). The main loop only reads a line, finds the command and calls the right function.
3. Store the data in containers that grow as needed (for example `std::vector`). Do not use a fixed-size array with a limit like 10.
4. Do not use global variables for the data if you can avoid it. Pass the data to your functions instead.
5. Use the **exact** prompts and messages given in this document, so that your instructor can test your program quickly.
6. The program must never crash or hang on bad input, including an empty line, letters where a number is expected, or an unknown command.

7. Comment your code: a short comment above each function that says what it does.
8. Work alone. Copied submissions receive zero.

7. Suggested plan: build it in stages

Do not write everything at once. Compile and test after every stage.

1. **Stage 1: the loop.** Print the banner, show the prompt, read a line, handle `help`, `exit` and unknown commands. Test case-insensitivity, spaces and empty lines.
2. **Stage 2: students.** Design the Student struct, then `add student` and `list students`.
3. **Stage 3: instructors.** The same for instructors, including the unique-name check.
4. **Stage 4: classes.** `create class` and `list classes`. Look up the instructor by name.
5. **Stage 5: enrolling.** `enroll` and `show class`. Think about where to store the link between a class and its students (for example, a list of student IDs inside the class).
6. **Stage 6: polish.** Test every error message and update the instructor's class count in `list instructors`.

Tips for beginners

- To read a full line with spaces, use `getline`. If you mix `cin >>` and `getline`, a leftover newline can make `getline` return an empty line. Read every input with `getline`, and convert text to numbers yourself.
- To split a line into words, a `stringstream` is the easiest tool.
- To compare text ignoring case, convert both texts to lowercase first (write a small helper function).
- To check that a text is a whole number, try converting it and make sure no characters are left over. Helper functions keep your code short.
- Search for a record with a simple loop that returns the position (or a "not found" value).

8. Sample full session

```
==========================================
 Student Management System   (type 'help')
==========================================
sms> add student
Full name: Ali Hassan
Age: 20
Email: ali@uni.edu
Student added successfully (ID: 1).
sms> add instructor
Full name: Dr. Sara Nabil
```

```
Department: Computer Science
Instructor added successfully (ID: 1).
sms> create class
Class name: Programming 101
Instructor name: dr. sara nabil
Capacity: 30
Class created successfully (ID: 1).
sms> enroll 1 1
Ali Hassan was enrolled in Programming 101.
sms> list classes
ID   Class Name       Instructor      Enrolled
---  ---------------  --------------  --------
1    Programming 101   Dr. Sara Nabil   1/30
Total: 1 class(es)
sms> exit
Goodbye!
```

Note that `dr. sara nabil` was accepted even though the instructor was added as `Dr. Sara Nabil`, and the class list shows the name as it was originally stored.

9. Grading (10 points)

Item	Points
Main loop, input parsing (case, spaces, empty line), `help`, `exit`, unknown command	1.5
`add student` with validation	1
`add instructor` with validation	1
`create class` with validation	1.5
`list students`, `list instructors`, `list classes`	1.5
`enroll` with all its checks	1.5
`show class`	0.5
Robustness: no crash on bad input, exact messages	1
Code organization (structs, one function per command, comments)	0.5
Total	**10**

10. Bonus (optional, up to +2)

Explain every bonus in a comment at the top of your main file.

1. **`unenroll <student_id> <class_id>`:** removes a student from a class. *(+0.5)*
2. **`remove student <id>`:** deletes a student and removes them from every class they were in. *(+0.5)*
3. **`search student <text>`:** lists the students whose name contains the text, ignoring case. *(+0.5)*
4. **`student classes <student_id>`:** lists all the classes a student is enrolled in. *(+0.5)*

11. Submission

- Submit all your `.cpp` and `.h` files, and nothing else (no compiled program).
- Your program must compile with `g++ -std=c++17 -Wall -Wextra *.cpp -o sms` without errors or warnings.
- Put your name and ID in a comment at the top of each file.
- Submit before the deadline announced by your instructor.

Good luck!